CURSO DE ESPECIALIZAÇÃO EM DEEP LEARNING

Tarefa 2 — Modelos de Classificação e Regressão com Redes Neuronais

Relatório de desenvolvimento, resultados e análise comparativa

Pedro Serrano · setembro de 2026

RESULTADOS PRINCIPAIS

CLASSIFICAÇÃO · FASHION-MNIST

89,2 %

acurácia no conjunto de teste (10 000 imagens)

Perda de teste (entropia cruzada): **0,313**
MLP 784 → 256 → 128 → 10, saída Softmax
235 146 parâmetros · 22 épocas de treino

REGRESSÃO · CALIFORNIA HOUSING

$R^2 = 0,80$

coeficiente de determinação no teste (4 128 zonas)

MAE: **0,340** (≈ 34 000 USD)
RMSE: **0,513** (≈ 51 300 USD)
MLP 8 → 128 → 64 → 32 → 1, saída linear
11 521 parâmetros · 94 épocas de treino

Este relatório acompanha o notebook *Tarefa2_Classificacao_Regressao_Pedro_Serrano.ipynb*, que contém o código comentado e todos os gráficos. As figuras, tabelas e capturas aqui apresentadas foram geradas a partir da execução desse notebook.

Python 3.12.6 · TensorFlow 2.21.0 · Keras 3.15.1 · scikit-learn 1.8.0 · NumPy 2.5.3 · pandas 3.0.6

Índice

1. Introdução **3**

2. Ambiente e metodologia **3**

3. Parte 1 — Modelo de classificação (Fashion-MNIST) **5**

3.1 Dataset e pré-processamento 5
3.2 Arquitetura 5
3.3 Compilação e treino 7
3.4 Evolução da perda e da acurácia 8
3.5 Avaliação no conjunto de teste 9
3.6 Discussão 11

4. Parte 2 — Modelo de regressão (California Housing) **12**

4.1 Dataset e justificação da escolha 12
4.2 Pré-processamento e modelos de referência 12
4.3 Arquitetura 13
4.4 Compilação e treino 14
4.5 Evolução da perda 15
4.6 Avaliação no conjunto de teste 15
4.7 Discussão 16

5. Parte 3 — Comparação e análise **17**

5.1 Diferenças de implementação 17
5.2 Diferenças no treino 17
5.3 Experiência: o efeito da escala dos dados 18
5.4 Desafios específicos de cada tipo de problema 19

6. Conclusões **21**

Anexo A — Capturas complementares do notebook **21**

1. Introdução

Esta tarefa tem como objetivo desenvolver, treinar e avaliar dois modelos de redes neuronais para os dois grandes tipos de problemas de aprendizagem supervisionada: um de **classificação**, em que o modelo escolhe uma classe entre várias possíveis, e um de **regressão**, em que o modelo prevê um valor numérico contínuo. No fim, comparam-se as duas implementações e discutem-se os desafios próprios de cada tipo de problema.

Os dois modelos são redes neuronais densas (MLP, *multilayer perceptron*), implementadas com TensorFlow/Keras, e partilham a mesma base: camadas ocultas densas com ativação ReLU, otimizador Adam e paragem antecipada. Mudam o formato dos dados, a camada de saída, a função de perda e as métricas, e são precisamente essas diferenças que este relatório procura explicar.

A Tabela 1 resume os requisitos do enunciado e a secção onde cada um é tratado. A secção 2 descreve o ambiente e a metodologia comum; as secções 3 e 4 apresentam cada modelo (dados, arquitetura, treino, curvas de aprendizagem e avaliação); a secção 5 compara-os e discute os desafios de cada tipo de problema; a secção 6 resume as conclusões. O anexo reúne capturas complementares do notebook.

***Tabela 1** — Requisitos do enunciado e onde são cumpridos*

Requisito do enunciado	Como foi cumprido	Secção
Parte 1 — dataset de classificação multiclasse	Fashion-MNIST (10 classes, 70 000 imagens)	3.1
Camada de entrada compatível com o dataset	`Input(shape=(28, 28))` e `Flatten` (784 valores)	3.2
Pelo menos duas camadas ocultas densas com ReLU	`Dense(256)` e `Dense(128)`, ambas com ReLU	3.2
Camada de saída com Softmax	`Dense(10, activation='softmax')`	3.2
Perda categórica	`sparse_categorical_crossentropy` (equivalência com a `categorical_crossentropy` verificada)	3.3
Gráficos de perda e acurácia; avaliação no teste	Figura 2; Tabela 5 e Tabela 6; Figura 3 e Figura 4	3.4–3.5
Parte 2 — dataset de regressão	California Housing (20 640 zonas, 8 atributos)	4.1
Pelo menos duas camadas ocultas densas com ReLU	`Dense(128)`, `Dense(64)` e `Dense(32)`, com ReLU	4.3
Saída com 1 neurónio e ativação linear	`Dense(1, activation='linear')`	4.3
Perda adequada (mse ou mae)	`mse`, com MAE e RMSE como métricas	4.4
Gráficos da perda; avaliação com MAE ou RMSE	Figura 6; Tabela 9 (MAE, RMSE e R^2); Figura 7	4.5–4.6
Parte 3 — comparação e desafios	Tabela 10 e Tabela 11; experiência sobre a escala dos dados; desafios de cada tipo	5
Entrega — notebook e relatório	Notebook comentado com os gráficos; este PDF com capturas, tabelas e análise	—

2. Ambiente e metodologia

O trabalho foi desenvolvido num notebook Jupyter, com Python 3.12.6, TensorFlow 2.21.0 (Keras 3.15.1) para as redes neuronais, scikit-learn 1.8.0 para a divisão dos dados, a padronização, os modelos de referência e as

métricas, pandas e NumPy para manipular os dados e Matplotlib para os gráficos. O treino correu em CPU: com redes deste tamanho, cada modelo treina em cerca de um minuto ou menos, sem necessidade de GPU.

Para os resultados serem reprodutíveis, fixei a semente aleatória (42) com `keras.utils.set_random_seed`, que controla a divisão dos dados, a inicialização dos pesos e o dropout. Mesmo assim, noutra máquina ou com outras versões das bibliotecas, os valores podem variar ligeiramente. Os dois modelos seguiram o mesmo procedimento:

1. Divisão dos dados em treino, validação e teste (Tabela 2). O conjunto de teste fica de parte até à avaliação final e é usado uma única vez.
2. Pré-processamento ajustado apenas com os dados de treino.
3. Construção da rede com a API `Sequential` do Keras e compilação (otimizador, função de perda e métricas).
4. Treino com mini-lotes, acompanhado no conjunto de validação, com paragem antecipada (`EarlyStopping`) e redução da taxa de aprendizagem quando a validação estagna (`ReduceLROnPlateau`).
5. Análise das curvas de aprendizagem e avaliação no conjunto de teste, com comparação com modelos de referência.

***Tabela 2** — Divisão dos dados e pré-processamento*

Dataset	Treino	Validação	Teste	Pré-processamento
Fashion-MNIST	54 000	6 000	10 000	Píxeis ÷ 255 (intervalo [0, 1])
California Housing	13 209	3 303	4 128	Padronização *z-score* (ajustada no treino)

3. Parte 1 — Modelo de classificação (Fashion-MNIST)

3.1 Dataset e pré-processamento

O Fashion-MNIST (Zalando, 2017) reúne 70 000 imagens de artigos de vestuário em tons de cinzento, com 28 × 28 píxeis e 10 classes equilibradas (Figura 1): 60 000 imagens de treino e 10 000 de teste. Escolhi-o em vez do MNIST porque é bastante mais exigente: um MLP simples passa dos 97 % de acurácia no MNIST, mas fica perto dos 88–90 % no Fashion-MNIST, o que deixa erros interessantes para analisar.



Figura 1 *— Um exemplo de cada uma das 10 classes do Fashion-MNIST (imagens de 28 × 28 píxeis).*

O pré-processamento teve dois passos. Os píxeis foram convertidos de inteiros entre 0 e 255 para reais entre 0 e 1 (divisão por 255), e 10 % das imagens de treino (6 000) foram separadas, de forma estratificada, para validação. As etiquetas ficaram como inteiros de 0 a 9, o que determina a escolha da função de perda (secção 3.3).

3.2 Arquitetura

A rede é um MLP com duas camadas ocultas densas (Tabela 3). A camada `Flatten` converte cada imagem num vetor de 784 valores, porque as camadas densas só recebem vetores. As camadas ocultas usam a ReLU, f(x) = max(0, x), que é barata de calcular e não satura para valores positivos, o que atenua o desvanecimento do gradiente. Depois de cada uma, um dropout de 20 % desliga neurónios ao acaso em cada passo de treino, para a rede não depender demasiado de neurónios específicos e sobreajustar menos. A saída tem 10 neurónios com Softmax, que converte os valores da última camada numa distribuição de probabilidades, uma por classe, que somam 1.

***Tabela 3** — Arquitetura do classificador (None = tamanho do mini-lote)*

Camada	Tipo	Forma da saída	Ativação	Parâmetros
entrada	Input	(None, 28, 28)	—	0
achatamento	Flatten	(None, 784)	—	0
oculta_1	Dense	(None, 256)	ReLU	200 960
dropout_1	Dropout	(None, 256)	—	0
oculta_2	Dense	(None, 128)	ReLU	32 896
dropout_2	Dropout	(None, 128)	—	0
saida	Dense	(None, 10)	Softmax	1 290
Total				**235 146**

Dos 235 146 parâmetros, 85 % estão na primeira camada oculta, que liga cada um dos 784 píxeis a cada um dos 256 neurónios (784 × 256 pesos + 256 vieses). É a camada mais pesada e a que mais contribui para o risco de sobreajuste. A Captura 1 mostra a definição do modelo no notebook e o resumo gerado pelo Keras.

```
[6]: def criar_classificador(taxa_dropout=0.2):
         """Cria o MLP de classificação: 784 entradas -> 256 -> 128 -> 10 classes."""
         return keras.Sequential([
             layers.Input(shape=(28, 28), name="entrada"),          # camada de entrada (imagem 28x28)
             layers.Flatten(name="achatamento"),                      # 28x28 -> vetor de 784 valores
             layers.Dense(256, activation="relu", name="oculta_1"),   # 1.ª camada oculta densa (ReLU)
             layers.Dropout(taxa_dropout, name="dropout_1"),          # regularização
             layers.Dense(128, activation="relu", name="oculta_2"),   # 2.ª camada oculta densa (ReLU)
             layers.Dropout(taxa_dropout, name="dropout_2"),          # regularização
             layers.Dense(10, activation="softmax", name="saida"),    # 10 probabilidades (Softmax)
         ], name="classificador_fashion_mnist")


     keras.utils.set_random_seed(SEMENTE)   # mesma inicialização dos pesos em cada execução
     modelo_clf = criar_classificador()
     modelo_clf.summary()
```

Model: "classificador_fashion_mnist"

Layer (type)	Output Shape	Param #
achatamento (Flatten)	(None, 784)	0
oculta_1 (Dense)	(None, 256)	200,960
dropout_1 (Dropout)	(None, 256)	0
oculta_2 (Dense)	(None, 128)	32,896
dropout_2 (Dropout)	(None, 128)	0
saida (Dense)	(None, 10)	1,290

Total params: 235,146 (918.54 KB)
Trainable params: 235,146 (918.54 KB)
Non-trainable params: 0 (0.00 B)

***Captura 1** — definição do classificador e resumo da arquitetura (`model.summary()`).*

3.3 Compilação e treino

Tabela 4 *— Configuração da compilação e do treino do classificador*

Hiperparâmetro	Valor
Otimizador	Adam, com taxa de aprendizagem inicial de 0,001
Função de perda	`sparse_categorical_crossentropy`
Métrica	`accuracy` (acurácia)
Mini-lote	128 imagens (422 atualizações dos pesos por época)
Épocas	Máximo de 50; paragem antecipada com paciência de 6 épocas (repõe os melhores pesos)
Taxa de aprendizagem	Reduzida para metade após 3 épocas sem melhoria (mínimo de 0,00001)
Regularização	Dropout de 0,2 depois de cada camada oculta

A entropia cruzada categórica mede a probabilidade que o modelo atribui à classe correta: para uma imagem da classe y, a perda é $-\log p_y$. Se o modelo der probabilidade 0,9 à classe certa, a perda é 0,11; se der 0,1, é 2,30. Como as etiquetas são inteiros, usei a versão *sparse*; com etiquetas em *one-hot*, usar-se-ia a `categorical_crossentropy`. As duas calculam o mesmo valor, o que verifiquei no notebook (Captura 2): com a rede ainda por treinar, ambas dão 2,3722, muito perto de ln(10) ≈ 2,3026, a perda de quem atribui a mesma probabilidade às 10 classes.

```
[7]: modelo_clf.compile(
         optimizer=keras.optimizers.Adam(learning_rate=1e-3),
         loss="sparse_categorical_crossentropy",   # etiquetas inteiras 0-9
         metrics=["accuracy"],
     )

     # Verificação: com as etiquetas em one-hot, a categorical_crossentropy dá o mesmo valor.
     # Com o modelo ainda por treinar, a perda deve andar perto de ln(10) ≈ 2.30, o valor de quem
     # atribui a mesma probabilidade às 10 classes (um palpite ao acaso).
     amostra_x, amostra_y = X_val_clf[:1000], y_val_clf[:1000]
     prob_iniciais = modelo_clf.predict(amostra_x, verbose=0)
     perda_sparse = float(keras.losses.SparseCategoricalCrossentropy()(amostra_y, prob_iniciais))
     perda_onehot = float(keras.losses.CategoricalCrossentropy()(
         keras.utils.to_categorical(amostra_y, 10), prob_iniciais))
     print(f"sparse_categorical_crossentropy (etiquetas inteiras): {perda_sparse:.4f}")
     print(f"categorical_crossentropy (etiquetas one-hot)        : {perda_onehot:.4f}")
     print(f"ln(10)                                              : {np.log(10):.4f}")

sparse_categorical_crossentropy (etiquetas inteiras): 2.3722
categorical_crossentropy (etiquetas one-hot)        : 2.3722
ln(10)                                              : 2.3026
```

Captura 2 *— compilação do classificador e verificação da equivalência entre as duas perdas categóricas.*

O treino durou 63 s em CPU. A Captura 3 mostra o registo: uma linha a cada 5 épocas, as reduções da taxa de aprendizagem e a paragem antecipada, que interrompeu o treino na época 22 e repôs os pesos da época 16.

```
[8]: callbacks_clf = [
         keras.callbacks.EarlyStopping(monitor="val_loss", patience=6, restore_best_weights=True, verbose=1),
         keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=3, min_lr=1e-5, verbose=1),
         ResumoEpocas(cada=5),
     ]

     inicio = time.perf_counter()
     historico_clf = modelo_clf.fit(
         X_treino_clf, y_treino_clf,
         validation_data=(X_val_clf, y_val_clf),   # avaliado no fim de cada época
         epochs=50, batch_size=128,
         callbacks=callbacks_clf, verbose=0)
     tempo_clf = time.perf_counter() - inicio

     hist_clf = historico_clf.history
     melhor_epoca_clf = int(np.argmin(hist_clf["val_loss"])) + 1
     print(f"\nTreino concluído em {tempo_clf:.0f} s | épocas executadas: {len(hist_clf['loss'])}"
           f" | melhor época: {melhor_epoca_clf}")

Época   1 | accuracy=0.7906  loss=0.5923  val_accuracy=0.8538  val_loss=0.4013
Época   5 | accuracy=0.8782  loss=0.3310  val_accuracy=0.8895  val_loss=0.3063
Época  10 | accuracy=0.8945  loss=0.2801  val_accuracy=0.8977  val_loss=0.2816

Epoch 12: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
Época  15 | accuracy=0.9142  loss=0.2270  val_accuracy=0.9012  val_loss=0.2723

Epoch 19: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
Época  20 | accuracy=0.9263  loss=0.1951  val_accuracy=0.9065  val_loss=0.2647

Epoch 22: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
Epoch 22: early stopping
Restoring model weights from the end of the best epoch: 16.

Treino concluído em 63 s | épocas executadas: 22 | melhor época: 16
```

Captura 3 *— treino do classificador, com os callbacks EarlyStopping e ReduceLROnPlateau.*

3.4 Evolução da perda e da acurácia

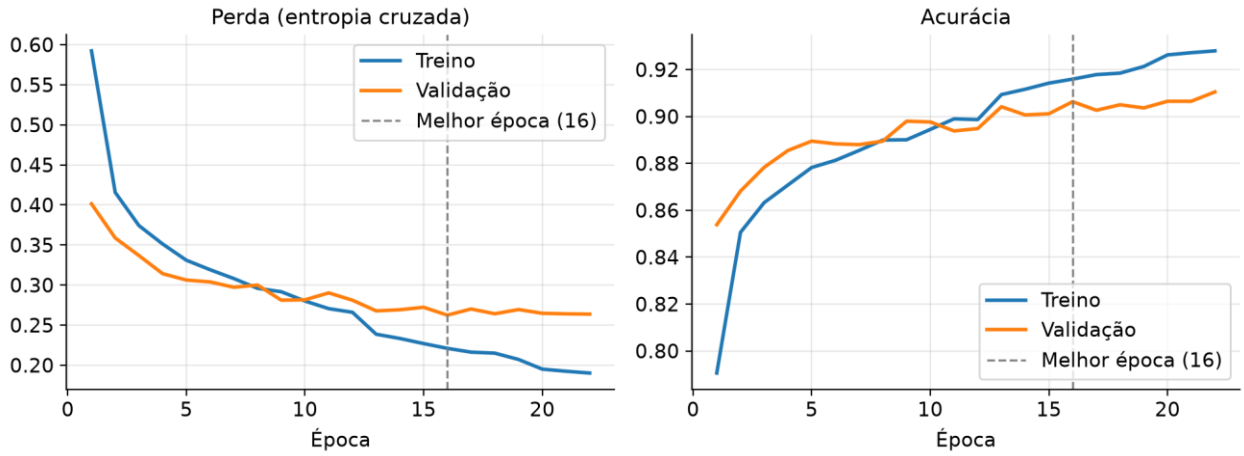


Figura 2 *— Evolução da perda (entropia cruzada) e da acurácia no treino e na validação. A linha tracejada marca a época com menor perda de validação, cujos pesos foram repostos.*

Leitura das curvas. Nas primeiras épocas, a perda desce depressa nos dois conjuntos, e a acurácia de validação chega aos 89 % logo na 5.ª época. A partir daí, a perda de treino continua a descer a bom ritmo, mas a de validação desce muito mais devagar e com oscilações. Por volta da 10.ª época, a perda de treino passa para baixo da de validação e a diferença entre as duas vai aumentando: é o início do sobreajuste (*overfitting*), em que a rede se ajusta a pormenores das imagens de treino que não se repetem noutras imagens. A taxa de aprendizagem foi reduzida para metade nas épocas 12 e 19, o que ajuda a afinar os pesos na fase final (a terceira redução, na época 22, coincidiu com a paragem). O mínimo da perda de validação (0,2624) foi atingido na época 16. Como não houve melhorias nas 6 épocas seguintes, o treino parou na época 22 e os pesos da época 16 foram repostos. Nessa época, a acurácia era de 91,6 % no treino e de 90,6 % na validação.

Nota: as métricas de treino são medidas com o dropout ativo e em média ao longo de cada época, enquanto a rede ainda está a aprender, por isso ficam um pouco pessimistas. Por isso, nas primeiras épocas (até à 9.ª), a perda de treino fica quase sempre acima da de validação.

3.5 Avaliação no conjunto de teste

O modelo final foi avaliado uma única vez no conjunto de teste (Tabela 5). A acurácia é acompanhada da precisão, do recall e do F1 de cada classe (Tabela 6), porque uma acurácia global esconde as classes em que o modelo falha mais.

***Tabela 5** — Métricas globais do classificador*

Métrica	Valor
Acurácia no teste	**89,16 %**
Perda (entropia cruzada) no teste	0,3134
Imagens mal classificadas	1 084 de 10 000
F1 médio (macro)	0,891
Acurácia de validação na melhor época	90,63 %
Referência: palpite ao acaso	10,00 %

***Tabela 6** — Métricas por classe no teste (a sombreado, as quatro classes com pior F1)*

Classe	Precisão	Recall	F1	Imagens
T-shirt/top	0,837	0,858	0,847	1 000
Calças	0,993	0,976	0,984	1 000
Camisola	0,815	0,792	0,803	1 000
Vestido	0,884	0,914	0,899	1 000
Casaco	0,803	0,833	0,817	1 000
Sandália	0,982	0,964	0,973	1 000
Camisa	0,729	0,682	0,705	1 000
Sapatilha	0,924	0,976	0,949	1 000
Mala	0,972	0,980	0,976	1 000
Botim	0,974	0,941	0,957	1 000
Média (macro)	0,891	0,892	0,891	10 000

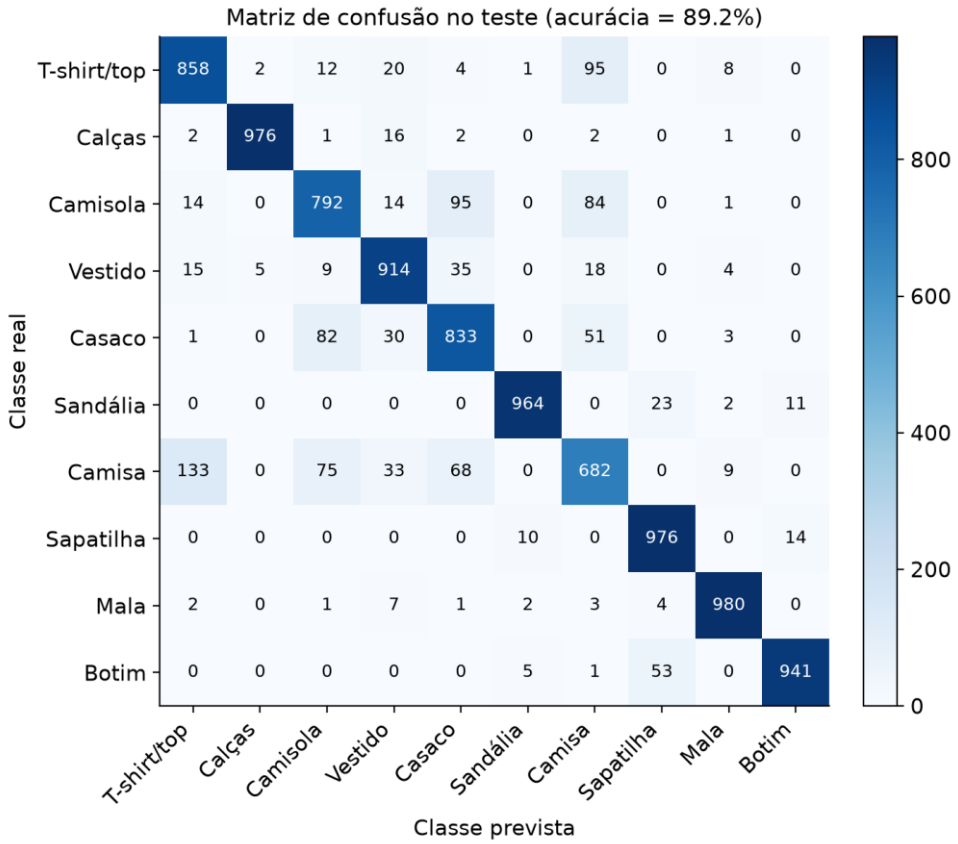


Figura 3 *— Matriz de confusão no conjunto de teste (1 000 imagens por classe). A diagonal conta os acertos.*



Figura 4 *— Exemplos de imagens de teste mal classificadas, com a classe real, a prevista e a confiança da previsão.*

A Figura 4 mostra alguns dos erros. Vários são difíceis mesmo para uma pessoa: com 28 × 28 píxeis, o que distingue uma camisa de uma T-shirt ou de uma camisola são pormenores como a gola, os botões ou o comprimento das mangas.

3.6 Discussão

- A rede acerta em **89,2 %** das 10 000 imagens de teste (1 084 erros), muito acima dos 10 % de um palpite ao acaso e dentro do que se espera de um MLP neste dataset. A acurácia de teste fica perto da de validação (90,6 %), o que mostra que a validação estimou bem o desempenho em imagens novas.
- Os erros não se distribuem ao acaso. As classes com pior F1 são Camisa (0,70), Camisola (0,80), Casaco (0,82) e T-shirt/top (0,85), todas peças de cima do corpo, com silhuetas muito parecidas a 28 × 28 píxeis. A classe mais difícil é **Camisa**: só 68 % das suas imagens são reconhecidas. As confusões mais frequentes são Camisa → T-shirt/top (133), Camisola → Casaco (95) e T-shirt/top → Camisa (95). No extremo oposto, Calças, Mala, Sandália e Botim têm F1 acima de 0,95.
- Parte desta limitação vem da arquitetura: o `Flatten` transforma a imagem numa lista de píxeis, e a rede perde a noção de vizinhança (contornos, texturas, formas). As redes convolucionais (CNN) exploram essa estrutura e costumam ultrapassar os 91–92 % neste dataset.
- Mesmo quando erra, a rede está muitas vezes confiante: a confiança média é de 93,6 % nas previsões certas, mas ainda de 68,6 % nas erradas. As probabilidades da Softmax não são, por isso, uma medida fiável da incerteza do modelo.

4. Parte 2 — Modelo de regressão (California Housing)

4.1 Dataset e justificação da escolha

O California Housing, incluído no scikit-learn, vem do censo dos EUA de 1990: 20 640 zonas censitárias da Califórnia, descritas por 8 atributos numéricos (rendimento mediano, idade mediana das casas, número médio de divisões e de quartos, população, ocupantes por habitação, latitude e longitude). O objetivo é prever o valor mediano das casas de cada zona, em centenas de milhares de dólares.

Preferi-o ao Boston Housing, sugerido no enunciado, por duas razões. A primeira é a dimensão: tem 20 640 exemplos, contra 506 no Boston Housing, e com tão poucos dados os resultados de uma rede neuronal variam muito com a divisão treino/teste e com a semente aleatória. A segunda é ética: o Boston Housing inclui uma variável construída a partir da proporção de população negra de cada zona e foi retirado do scikit-learn na versão 1.2 por esse motivo. O California Housing é o substituto recomendado.

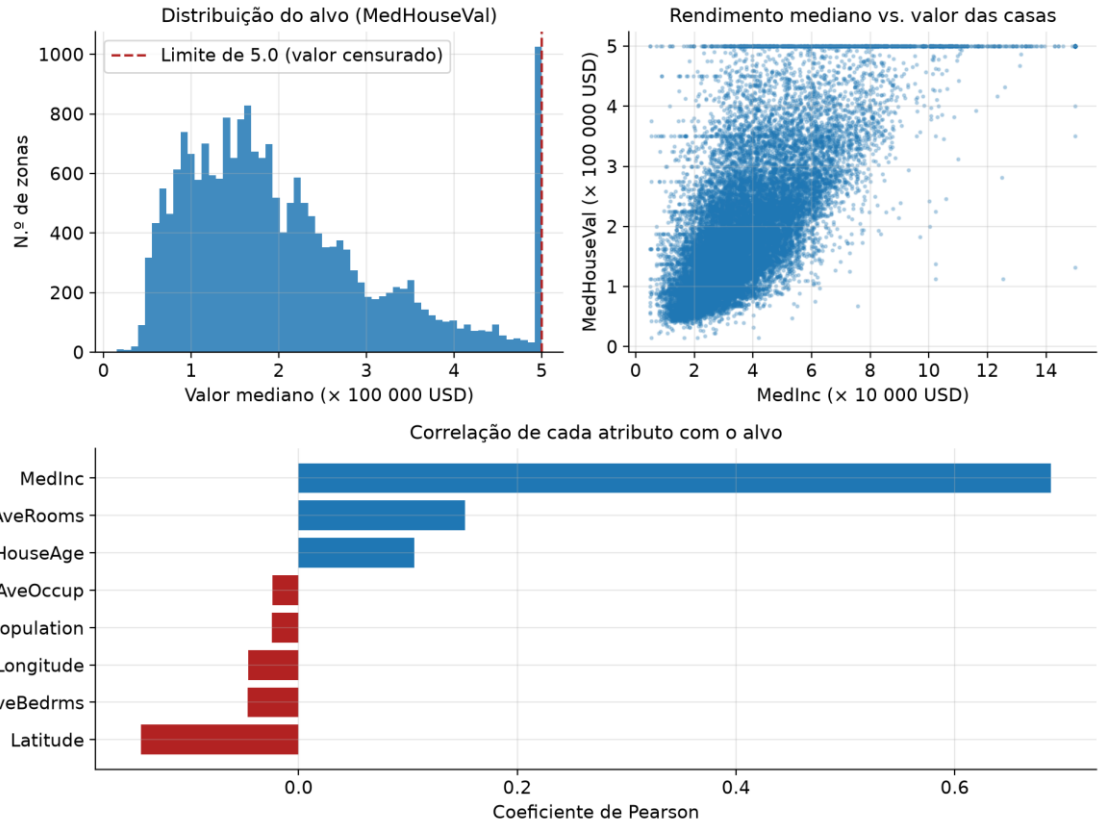


Figura 5 *— Exploração do California Housing: distribuição do alvo, com o limite de 5,0; rendimento mediano vs. valor das casas; correlação linear de cada atributo com o alvo.*

A Figura 5 mostra três aspetos que condicionam o problema. O alvo tem uma distribuição assimétrica e um pico artificial em 5,0: no censo, as casas de 500 000 USD ou mais foram registadas com esse valor, o que acontece em 992 zonas (4,8 %). O rendimento mediano é o atributo mais correlacionado com o preço (r = 0,69), mas a relação é ruidosa e o limite de 5,0 aparece como uma linha horizontal. Os restantes atributos têm correlações lineares fracas, o que não quer dizer que sejam inúteis: a latitude e a longitude, por exemplo, só ganham significado em conjunto, e esse tipo de relação não linear é o que uma rede neuronal consegue aprender.

4.2 Pré-processamento e modelos de referência

Dividi os dados em 64 % para treino, 16 % para validação e 20 % para teste (Tabela 2). Como os atributos têm escalas muito diferentes (a população chega às dezenas de milhares e o número médio de quartos anda perto

de 1), padronizei-os com o `StandardScaler`: cada atributo passa a ter média 0 e desvio-padrão 1. O escalador foi ajustado apenas com os dados de treino e depois aplicado à validação e ao teste; ajustá-lo com todos os dados deixaria passar informação do teste para o treino (*data leakage*). O alvo manteve a escala original (0,15 a 5,0), que já é pequena, para as métricas saírem diretamente em centenas de milhares de dólares.

Um MAE ou um RMSE isolados dizem pouco. Para interpretar os resultados, treinei também dois modelos de referência (*baselines*): um que prevê sempre a média do treino, que é o que se consegue sem olhar para os atributos, e uma regressão linear com os mesmos dados padronizados.

4.3 Arquitetura

A rede tem três camadas ocultas densas com ReLU, em forma de funil (128 → 64 → 32), e uma saída com um único neurónio de ativação linear (Tabela 7). A saída linear é o que faz desta rede um regressor: devolve um número real sem limites, enquanto uma Sigmoide limitaria a previsão ao intervalo]0, 1[e uma Softmax com um só neurónio devolveria sempre 1. Com apenas 8 entradas, a rede é muito mais pequena do que a de classificação (11 521 parâmetros, contra 235 146). Não usei dropout: com tão poucos parâmetros, a paragem antecipada foi suficiente, porque a perda de validação estabiliza sem voltar a subir (secção 4.5).

***Tabela 7** — Arquitetura do regressor (None = tamanho do mini-lote)*

Camada	Tipo	Forma da saída	Ativação	Parâmetros
`entrada`	Input	`(None, 8)`	—	0
`oculta_1`	Dense	`(None, 128)`	ReLU	1 152
`oculta_2`	Dense	`(None, 64)`	ReLU	8 256
`oculta_3`	Dense	`(None, 32)`	ReLU	2 080
`saida`	Dense	`(None, 1)`	Linear	33
Total				**11 521**

```
[17]: def criar_regressor(n_atributos):
          """Cria o MLP de regressão: n_atributos -> 128 -> 64 -> 32 -> 1 valor."""
          return keras.Sequential([
              layers.Input(shape=(n_atributos,), name="entrada"),     # camada de entrada (8 atributos)
              layers.Dense(128, activation="relu", name="oculta_1"),  # 1.ª camada oculta densa (ReLU)
              layers.Dense(64, activation="relu", name="oculta_2"),   # 2.ª camada oculta densa (ReLU)
              layers.Dense(32, activation="relu", name="oculta_3"),   # 3.ª camada oculta densa (ReLU)
              layers.Dense(1, activation="linear", name="saida"),     # 1 neurónio, ativação linear
          ], name="regressor_california_housing")


      keras.utils.set_random_seed(SEMENTE)
      modelo_reg = criar_regressor(X_treino_reg_p.shape[1])
      modelo_reg.summary()
```

Model: "regressor_california_housing"

Layer (type)	Output Shape	Param #
oculta_1 (Dense)	(None, 128)	1,152
oculta_2 (Dense)	(None, 64)	8,256
oculta_3 (Dense)	(None, 32)	2,080
saida (Dense)	(None, 1)	33

Total params: 11,521 (45.00 KB)
Trainable params: 11,521 (45.00 KB)
Non-trainable params: 0 (0.00 B)

***Captura 4** — definição do regressor e resumo da arquitetura.*

4.4 Compilação e treino

***Tabela 8** — Configuração da compilação e do treino do regressor*

Hiperparâmetro	Valor
Otimizador	Adam, com taxa de aprendizagem inicial de 0,001
Função de perda	`mse` (erro quadrático médio)
Métricas	`mae` e `RootMeanSquaredError` (RMSE)
Mini-lote	64 exemplos (207 atualizações dos pesos por época)
Execução	`steps_per_execution=32`: 32 mini-lotes por chamada ao TensorFlow (mais rápido, mesmos resultados)
Épocas	Máximo de 300; paragem antecipada com paciência de 20 épocas (repõe os melhores pesos)
Taxa de aprendizagem	Reduzida para metade após 8 épocas sem melhoria (mínimo de 0,00001)
Regularização	Apenas a paragem antecipada

O MSE, $(1/n)\cdot\Sigma(y - \hat{y})^2$, penaliza os erros de forma quadrática: um erro de 1,0 pesa 100 vezes mais do que um erro de 0,1. É a perda habitual em regressão e leva o modelo a estimar o valor médio do alvo. Como métricas, acompanhei o MAE, que mede o erro médio em valor absoluto e é o mais fácil de interpretar, e o RMSE, a raiz do MSE, que fica na mesma unidade do alvo mas dá mais peso aos erros grandes. A paciência é maior do que na classificação porque as curvas de regressão são mais ruidosas e melhoram mais devagar. Como cada passo desta rede é muito leve, usei também `steps_per_execution=32`, que junta 32 mini-lotes em cada chamada ao TensorFlow: o treino fica bastante mais rápido e os resultados não mudam. O treino durou 26 s em CPU (Captura 5).

```
[19]: callbacks_reg = [
          keras.callbacks.EarlyStopping(monitor="val_loss", patience=20, restore_best_weights=True, verbose=1),
          keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=8, min_lr=1e-5, verbose=1),
          ResumoEpocas(cada=20),
      ]

      inicio = time.perf_counter()
      historico_reg = modelo_reg.fit(
          X_treino_reg_p, y_treino_reg,
          validation_data=(X_val_reg_p, y_val_reg),
          epochs=300, batch_size=64,
          callbacks=callbacks_reg, verbose=0)
      tempo_reg = time.perf_counter() - inicio

      hist_reg = historico_reg.history
      melhor_epoca_reg = int(np.argmin(hist_reg["val_loss"])) + 1
      print(f"\nTreino concluído em {tempo_reg:.0f} s | épocas executadas: {len(hist_reg['loss'])}"
            f" | melhor época: {melhor_epoca_reg}")

Época   1 | loss=0.9047  mae=0.6426  rmse=0.9511  val_loss=0.4461  val_mae=0.4667  val_rmse=0.6679
Época  20 | loss=0.2603  mae=0.3474  rmse=0.5102  val_loss=0.2970  val_mae=0.3867  val_rmse=0.5449
Época  40 | loss=0.2282  mae=0.3255  rmse=0.4777  val_loss=0.2826  val_mae=0.3704  val_rmse=0.5316

Epoch 53: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
Época  60 | loss=0.2014  mae=0.3068  rmse=0.4488  val_loss=0.2778  val_mae=0.3579  val_rmse=0.5271

Epoch 63: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.

Epoch 72: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.
Época  80 | loss=0.1833  mae=0.2920  rmse=0.4281  val_loss=0.2712  val_mae=0.3445  val_rmse=0.5208

Epoch 82: ReduceLROnPlateau reducing learning rate to 6.25000029685907e-05.

Epoch 90: ReduceLROnPlateau reducing learning rate to 3.125000148429535e-05.
Epoch 94: early stopping
Restoring model weights from the end of the best epoch: 74.

Treino concluído em 26 s | épocas executadas: 94 | melhor época: 74
```

***Captura 5** — compilação e treino do regressor.*

4.5 Evolução da perda

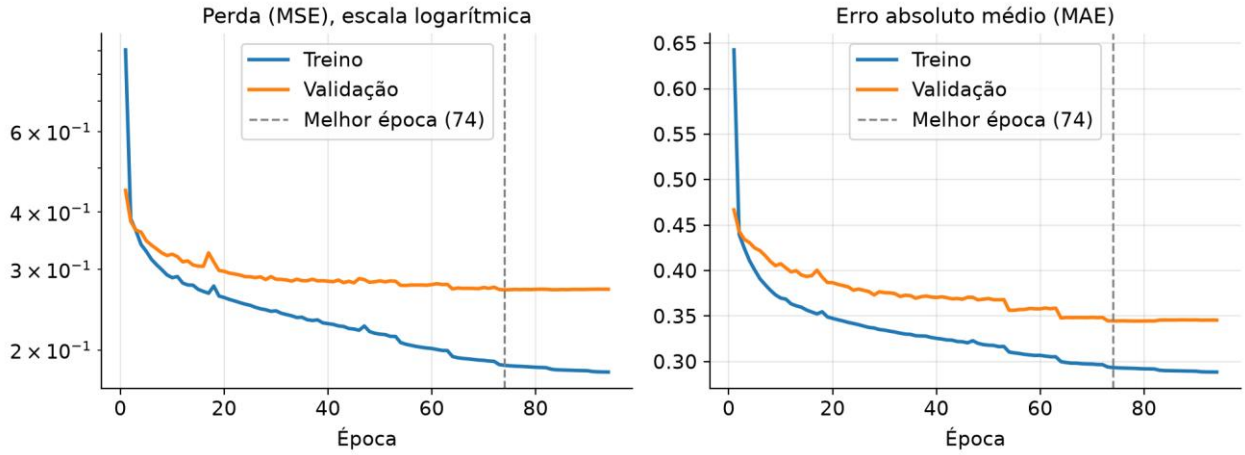


***Figura 6** — Evolução da perda (MSE, em escala logarítmica) e do MAE no treino e na validação. A linha tracejada marca a época com menor perda de validação.*

Leitura das curvas. A perda cai muito nas primeiras épocas (o MSE de validação passa de 0,446 na 1.ª época para 0,324 na 10.ª) e depois desce devagar. A partir da época 48, aproximadamente, a perda de validação fica a menos de 5 % do seu mínimo e quase não melhora, enquanto a de treino continua a descer: a rede começa a ajustar-se a particularidades dos dados de treino. O sobreajuste é ligeiro, porque a perda de validação estabiliza sem voltar a subir; na melhor época (74), o MAE é de 0,293 no treino e de 0,345 na validação. A curva de validação oscila mais do que a de treino, porque a perda de treino é uma média ao longo de toda a época, enquanto a de validação é medida num único ponto, no fim de cada época, e reflete o "ruído" dos passos de otimização. As 5 reduções da taxa de aprendizagem atenuam essas oscilações na fase final. O treino parou na época 94, 20 épocas depois do mínimo da perda de validação.

4.6 Avaliação no conjunto de teste

A Tabela 9 compara a rede com os dois modelos de referência no conjunto de teste, e a Figura 7 analisa os erros: os valores previstos face aos reais, a distribuição dos resíduos e o MAE por faixa de valor real.

***Tabela 9** — Métricas de regressão no conjunto de teste (valores em centenas de milhares de USD)*

Modelo	MAE	RMSE	R^2	MAE em USD
Média do treino (referência)	0,905	1,145	-0,000	90 500 USD
Regressão linear (referência)	0,533	0,745	0,577	53 300 USD
Rede neuronal (MLP)	**0,340**	**0,513**	**0,800**	**34 000 USD**

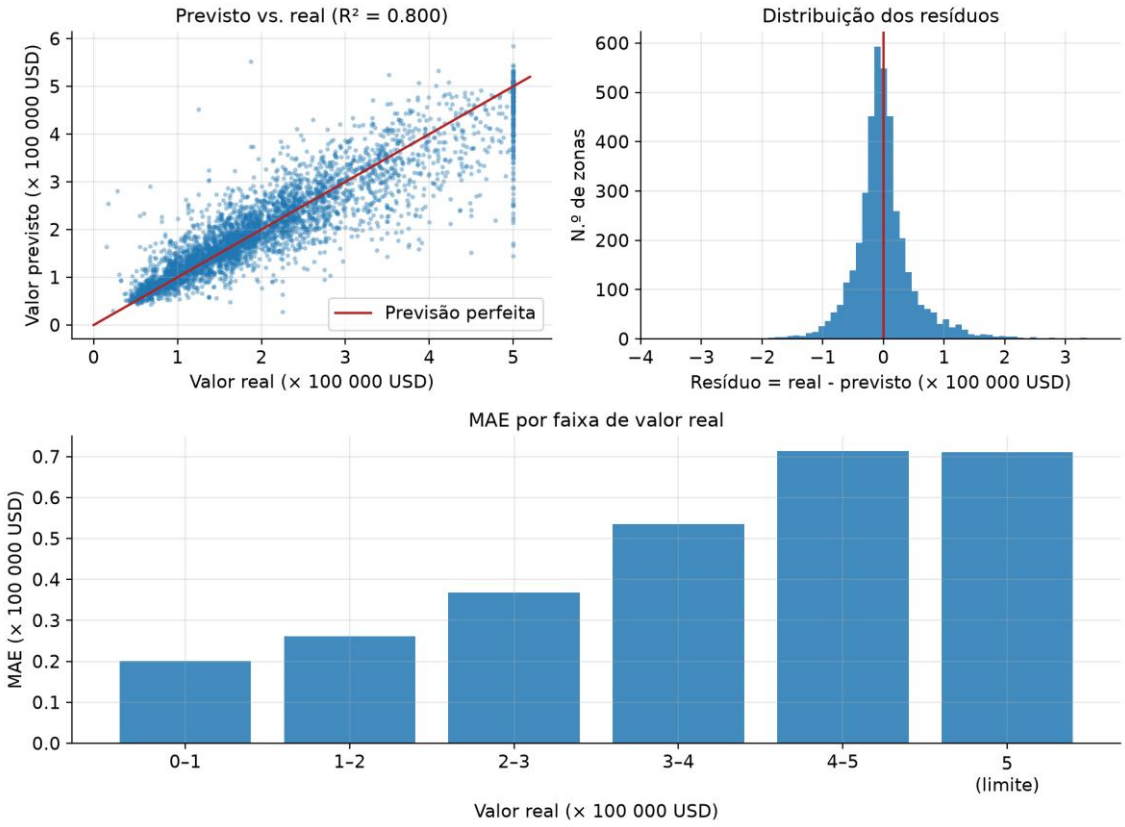


Figura 7 — Avaliação do regressor no teste: valores previstos vs. reais, distribuição dos resíduos e MAE por faixa de valor real.

4.7 Discussão

- No teste, a rede tem um MAE de **0,340** (≈ 34 000 USD) e um RMSE de **0,513** (≈ 51 300 USD), com **R^2 = 0,800**: explica cerca de 80 % da variância dos preços, contra 58 % da regressão linear, e reduz o RMSE em 31 % e o MAE em 36 % face a esta. A diferença mostra que a relação entre os atributos e o preço não é linear: o efeito da localização, por exemplo, depende da combinação da latitude com a longitude, e a rede aprende-o sem ser preciso criar esses atributos à mão.
- Os resíduos estão centrados perto de zero (média de 0,002), mas o erro cresce com o valor das casas: o MAE passa de 0,200 na faixa 0–1 para 0,714 na faixa 4–5 e mantém-se nesse nível (0,711) nas 184 zonas de teste com o valor no limite de 5,0. O censo registou "500 000 USD ou mais" como 5,0, por isso o valor verdadeiro destas zonas é desconhecido, e a rede tende a subestimá-lo (no gráfico previsto vs. real, a maioria dos pontos em x = 5 fica abaixo da diagonal).
- A saída linear não tem limites: as previsões vão de 0,273 a 5,849, e as mais altas ultrapassam o máximo registado nos dados (5,0). Para as zonas censuradas, isso até pode estar mais perto do valor real, mas mostra que a rede não conhece as restrições do problema; se fossem necessárias, teriam de ser impostas à parte (por exemplo, limitando as previsões ao intervalo válido).

5. Parte 3 — Comparação e análise

5.1 Diferenças de implementação

Os dois modelos usam os mesmos blocos: camadas densas com ReLU, otimizador Adam, retropropagação do erro e paragem antecipada. O que muda está à volta desse núcleo, como resume a Tabela 10: o formato do alvo, a camada de saída, a função de perda e a forma de avaliar. A camada de saída e a perda têm de ser escolhidas em conjunto. A Softmax produz probabilidades, e a entropia cruzada mede quão bem essas probabilidades acertam na classe; a saída linear produz um valor livre, e o MSE mede a distância a que esse valor fica do real.

***Tabela 10** — Diferenças de implementação entre os dois modelos*

Aspeto	Classificação (Fashion-MNIST)	Regressão (California Housing)
O que se prevê	Uma de 10 classes (valor discreto)	Um valor contínuo (preço mediano)
Entrada	Imagem 28 × 28 → `Flatten` → 784 valores	Vetor de 8 atributos numéricos
Pré-processamento	Píxeis ÷ 255 → [0, 1]	Padronização *z-score*, ajustada só no treino
Formato do alvo	Inteiros de 0 a 9	Real entre 0,15 e 5,0 (× 100 000 USD)
Camada de saída	`Dense(10, activation='softmax')`	`Dense(1, activation='linear')`
O que a saída devolve	10 probabilidades que somam 1	Um número real, sem limites
Função de perda	`sparse_categorical_crossentropy`	`mse` (erro quadrático médio)
Métricas	Acurácia; precisão, recall e F1 por classe	MAE, RMSE e R^2
Leitura da previsão	Classe com maior probabilidade (argmax)	O próprio valor (× 100 000 USD)
Diagnóstico dos erros	Matriz de confusão; exemplos mal classificados	Resíduos; previsto vs. real; erro por faixa de valor
Referência mínima	10 % (palpite ao acaso)	Prever sempre a média; regressão linear
Regularização	Dropout (0,2) e paragem antecipada	Paragem antecipada

5.2 Diferenças no treino

A Tabela 11 resume o treino dos dois modelos, que diferiu em quatro pontos.

***Tabela 11** — Comparação do treino dos dois modelos*

	Classificação	Regressão
Exemplos de treino	54 000	13 209
Parâmetros treináveis	235 146	11 521
Mini-lote / atualizações por época	128 / 422	64 / 207
Épocas executadas (melhor época)	22 (16)	94 (74)
Total de atualizações dos pesos	9 284	19 458
Tempo de treino (CPU)	63 s	26 s
Tempo médio por época	2,86 s	0,28 s
Perda de validação: 1.ª época → mínimo	0,401 → 0,262	0,446 → 0,271
Treino vs. validação na melhor época	acurácia 91,6 % vs. 90,6 %	MAE 0,293 vs. 0,345

1. **As perdas não são comparáveis entre si.** A entropia cruzada mede probabilidades (começa perto de ln 10 ≈ 2,30 e desceu até 0,262 na validação), e o MSE mede erros ao quadrado na unidade do alvo (desceu até 0,271). Valores parecidos não querem dizer o mesmo; o que se compara são as métricas de cada tarefa face a uma referência.
2. **Velocidade de convergência.** A classificação precisou de menos épocas (22 contra 94), em parte porque cada época tem mais exemplos (54 000 contra 13 209) e mais atualizações dos pesos (422 contra 207). A regressão fez mais épocas, com mais paciência, e a sua validação oscilou mais. Em tempo, a ordem inverte-se: cada época de classificação é muito mais pesada, e o treino demorou 63 s, contra 26 s na regressão.
3. **Sobreajuste.** Com 235 mil parâmetros e entradas de 784 dimensões, a rede de classificação começa a sobreajustar ao fim de poucas épocas, mesmo com dropout, e a diferença entre treino e validação vai aumentando. A rede de regressão, com 11,5 mil parâmetros e 8 entradas, também se afasta um pouco (MAE de 0,293 no treino contra 0,345 na validação), mas a sua perda de validação estabiliza sem voltar a subir, e a paragem antecipada basta.
4. **Relação entre perda e métrica.** Na regressão, a perda e as métricas andam juntas (o RMSE é a raiz do MSE). Na classificação, a acurácia é descontínua: só muda quando uma previsão passa de certa a errada, e pode ficar estável enquanto a perda sobe, porque a rede fica mais confiante nos erros que já comete.

5.3 Experiência: o efeito da escala dos dados

Nos dois problemas, pus as entradas numa escala pequena antes do treino. Para medir a importância deste passo, treinei as mesmas arquiteturas com e sem ele, com a mesma inicialização dos pesos, um número fixo de épocas (10 na classificação, 100 na regressão) e sem callbacks, para a comparação ser justa (Figura 8 e Tabela 12).

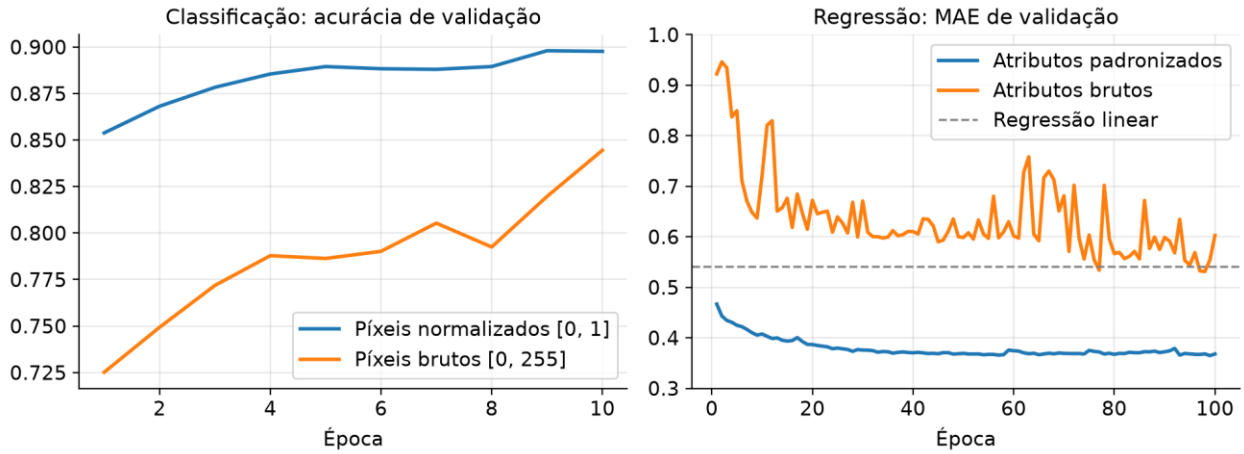


***Figura 8** — Efeito da escala dos dados: acurácia de validação do classificador e MAE de validação do regressor, com e sem normalização/padronização. A linha tracejada é o MAE de validação da regressão linear.*

***Tabela 12** — Resultados da experiência sobre a escala dos dados (conjunto de validação)*

Métrica de validação	Com escala adequada	Dados brutos
Classificação: melhor acurácia (10 épocas)	89,80 %	84,45 %
Classificação: acurácia na última época	89,77 %	84,45 %
Regressão: melhor MAE (100 épocas)	0,365	0,531
Regressão: MAE na última época	0,368	0,603
Referência: regressão linear (MAE)	0,541	—

Resultado. Sem a escala adequada, os dois modelos pioram:

- **Classificação:** com os píxeis entre 0 e 255, a melhor acurácia de validação desce de 89,8 % para 84,5 %, ou seja, a taxa de erro aumenta mais de 50 %. O Adam adapta o passo de cada peso e compensa parte do problema, mas as somas ponderadas da primeira camada ficam 255 vezes maiores e o treino torna-se mais lento e instável (curva laranja do gráfico).
- **Regressão:** com os atributos brutos, o melhor MAE de validação sobe de 0,365 para 0,531 (+46 %), ao nível da regressão linear (0,541), e na última época fica mesmo pior do que ela (0,603), apesar de a rede ser muito mais flexível. Os atributos com valores grandes, como a `Population` (milhares), dominam as somas ponderadas e abafam o `MedInc` (unidades), que é o mais informativo. Com entradas em escalas tão diferentes, a superfície da perda fica muito alongada e o otimizador avança aos ziguezagues, o que explica as oscilações da curva.

A normalização não é um pormenor: sobretudo em regressão, com atributos em escalas muito diferentes, pode pesar mais do que qualquer ajuste à arquitetura.

5.4 Desafios específicos de cada tipo de problema

Classificação

- **Classes visualmente semelhantes.** Os erros concentram-se num pequeno grupo de classes (Camisa, Camisola, Casaco e T-shirt/top). Com 28 × 28 píxeis e sem noção de vizinhança espacial, o MLP tem dificuldade em distinguir pormenores como golas, botões ou o comprimento das mangas. É um limite da arquitetura, que as redes convolucionais resolvem em boa parte.
- **Perda e acurácia medem coisas diferentes.** A acurácia só verifica se a classe mais provável é a certa; a entropia cruzada também mede a confiança. A perda de validação pode subir, porque a rede fica demasiado confiante nos erros, enquanto a acurácia se mantém. Por isso, a paragem antecipada vigia a perda e não a acurácia.
- **Probabilidades pouco calibradas.** Nas previsões erradas, a confiança média ainda é de 68,6 %. Se a decisão dependesse da confiança (por exemplo, pedir revisão humana abaixo de um limiar), seria preciso calibrar as probabilidades.
- **Formato das etiquetas e equilíbrio das classes.** A perda tem de corresponder ao formato das etiquetas: inteiros com a `sparse_categorical_crossentropy`, *one-hot* com a `categorical_crossentropy`; trocá-las dá erro de dimensões ou resultados errados. Aqui as classes são equilibradas; num problema desequilibrado, a acurácia engana, e é preciso olhar para o F1 de cada classe ou dar pesos diferentes às classes.
- **Muitos parâmetros para imagens pequenas.** Ligar 784 píxeis a 256 neurónios cria 200 mil pesos logo na primeira camada, o que favorece o sobreajuste; daí o dropout e a paragem antecipada.

Regressão

- **Escala dos atributos e do alvo.** A experiência da secção 5.3 mostrou que, sem padronização, a rede treina muito pior. A escala do alvo também conta: com o preço em dólares (até 500 000), o MSE seria da ordem de 10^{10} e os gradientes iniciais enormes, o que obrigaria a ajustar a taxa de aprendizagem ou a normalizar o alvo.
- **Não há uma acurácia.** O MAE e o RMSE dependem da unidade do alvo e só ganham significado quando comparados com uma referência (a média, a regressão linear); o R^2 ajuda a comparar entre problemas. As métricas também têm de ser lidas no contexto: um erro médio de cerca de 34 000 USD pode ser aceitável para uma estimativa rápida, mas não para uma avaliação bancária.

- **Sensibilidade a valores extremos.** O MSE eleva os erros ao quadrado, e algumas zonas atípicas, com médias de divisões ou de ocupantes por habitação muito elevadas (a AveOccup chega a 1 243), pesam muito no treino. O MAE ou a perda de Huber são alternativas mais robustas.
- **Qualidade do alvo.** O valor de 5,0 é um limite e não o preço real; o erro concentra-se nessas zonas, e nenhuma arquitetura corrige um alvo mal medido.
- **Saída sem restrições e erro não uniforme.** A saída linear pode devolver valores impossíveis, e o erro cresce com o valor das casas. Prever o logaritmo do preço ajuda nos dois casos.
- **Treino mais ruidoso.** A validação oscila mais, e o modelo precisa de mais épocas e de mais paciência na paragem antecipada.

Comuns aos dois

- **Sobreajuste e escolha de hiperparâmetros.** O número de camadas e de neurónios, a taxa de aprendizagem, o tamanho do mini-lote e o dropout não têm valores universais: cada escolha tem de ser validada no conjunto de validação, nunca no de teste.
- **Fugas de informação (*data leakage*).** Qualquer transformação ajustada aos dados, como o escalador, deve usar só o treino.
- **Reprodutibilidade.** A inicialização aleatória e a ordem dos mini-lotes mudam os resultados de execução para execução. Fixar a semente torna a comparação justa, mas uma avaliação rigorosa pediria várias execuções ou validação cruzada.

6. Conclusões

1. Os dois modelos cumprem os requisitos do enunciado e têm bons resultados para redes densas simples: **89,2 %** de acurácia no Fashion-MNIST e **R^2 = 0,800** (MAE ≈ 34 000 USD) no California Housing, bem acima das referências (10 % num palpite ao acaso; R^2 = 0,577 na regressão linear).
2. A mesma "máquina" serve os dois problemas: camadas densas com ReLU, otimizador Adam e retropropagação. O que define o tipo de problema é a **camada de saída** (Softmax ou linear), a **função de perda** (entropia cruzada ou MSE) e as **métricas** usadas para avaliar.
3. O pré-processamento pesa tanto como a arquitetura: sem normalizar ou padronizar os dados, os dois modelos pioram e a rede de regressão perde a vantagem que tinha sobre a regressão linear.
4. Os erros que restam têm causas diferentes. Na classificação, vêm sobretudo da arquitetura: o MLP não vê a estrutura espacial das imagens, e uma CNN é o passo natural seguinte. Na regressão, vêm sobretudo dos dados: o alvo está censurado a 5,0 e há zonas atípicas. Aí, as melhorias passam por tratar os dados (prever o logaritmo do preço, usar a perda de Huber) e por validação cruzada, mais do que por aumentar a rede.

Como continuação, testaria uma rede convolucional no Fashion-MNIST e, no California Housing, a previsão do logaritmo do preço, a perda de Huber e uma pesquisa sistemática de hiperparâmetros (por exemplo, com o KerasTuner) avaliada por validação cruzada.

Anexo A — Capturas complementares do notebook

Estas capturas mostram as saídas de avaliação tal como aparecem no notebook. Os mesmos valores estão organizados nas tabelas do relatório.

```
[10]: res_clf = modelo_clf.evaluate(X_teste_clf, y_teste_clf, verbose=0, return_dict=True)
      print(f"Perda no teste   : {res_clf['loss']:.4f}")
      print(f"Acurácia no teste: {res_clf['accuracy']:.4f}  ({res_clf['accuracy'] * 100:.2f} %)\n")

      # A rede devolve 10 probabilidades por imagem; a classe prevista é a de maior probabilidade
      prob_teste = modelo_clf.predict(X_teste_clf, verbose=0)
      y_prev_clf = prob_teste.argmax(axis=1)

      # Precisão, recall (sensibilidade) e F1 de cada classe
      relatorio_clf = classification_report(y_teste_clf, y_prev_clf, target_names=NOMES_CLASSES,
                                            digits=3, output_dict=True)
      print(classification_report(y_teste_clf, y_prev_clf, target_names=NOMES_CLASSES, digits=3))
```

```
Perda no teste   : 0.3134
Acurácia no teste: 0.8916  (89.16 %)

              precision    recall  f1-score   support

 T-shirt/top      0.837     0.858     0.847      1000
      Calças      0.993     0.976     0.984      1000
    Camisola      0.815     0.792     0.803      1000
     Vestido      0.884     0.914     0.899      1000
      Casaco      0.803     0.833     0.817      1000
    Sandália      0.982     0.964     0.973      1000
      Camisa      0.729     0.682     0.705      1000
   Sapatilha      0.924     0.976     0.949      1000
        Mala      0.972     0.980     0.976      1000
       Botim      0.974     0.941     0.957      1000

    accuracy                          0.892     10000
   macro avg      0.891     0.892     0.891     10000
weighted avg      0.891     0.892     0.891     10000
```

***Captura 6** — avaliação do classificador no conjunto de teste (`classification_report`).*

[21]:
```
# Avaliação com o Keras: perda (MSE) e métricas definidas na compilação
res_reg = modelo_reg.evaluate(X_teste_reg_p, y_teste_reg, verbose=0, return_dict=True)
print(f"Keras -> MSE: {res_reg['loss']:.4f} | MAE: {res_reg['mae']:.4f} | RMSE: {res_reg['rmse']:.4f}")

# As mesmas métricas com o scikit-learn, a partir das previsões, mais o R²
prev_rede = modelo_reg.predict(X_teste_reg_p, verbose=0).ravel()
metricas_rede = metricas_regressao(y_teste_reg, prev_rede)

tabela_reg = pd.concat([referencias, pd.DataFrame({"Rede neuronal (MLP)": metricas_rede}).T])
tabela_reg["MAE (USD)"] = (tabela_reg["MAE"] * 100_000).round(-2).astype(int)
tabela_reg["RMSE (USD)"] = (tabela_reg["RMSE"] * 100_000).round(-2).astype(int)
display(tabela_reg.round(4))

melhoria_rmse = 1 - metricas_rede["RMSE"] / referencias.loc["Regressão linear", "RMSE"]
melhoria_mae = 1 - metricas_rede["MAE"] / referencias.loc["Regressão linear", "MAE"]
print(f"Face à regressão linear, a rede reduz o RMSE em {melhoria_rmse:.1%} e o MAE em {melhoria_mae:.1%}.")
print(f"Previsões da rede entre {prev_rede.min():.3f} e {prev_rede.max():.3f} "
      f"(valores reais entre {y_teste_reg.min():.3f} e {y_teste_reg.max():.3f}).")
```

```
Keras -> MSE: 0.2627 | MAE: 0.3404 | RMSE: 0.5125
```

	MAE	RMSE	R²	MAE (USD)	RMSE (USD)
Média do treino	0.9051	1.1448	-0.0001	90500	114500
Regressão linear	0.5335	0.7446	0.5769	53300	74500
Rede neuronal (MLP)	0.3404	0.5125	0.7995	34000	51300

```
Face à regressão linear, a rede reduz o RMSE em 31.2% e o MAE em 36.2%.
Previsões da rede entre 0.273 e 5.849 (valores reais entre 0.150 e 5.000).
```

Captura 7 *— avaliação do regressor no teste e comparação com os modelos de referência.*

[23]:
```
import math

resumo = pd.DataFrame({
    "Classificação (Fashion-MNIST)": [
        "10 classes (inteiros 0-9)",
        f"{fmt(len(X_treino_clf))} / {fmt(len(X_val_clf))} / {fmt(len(X_teste_clf))}",
        "784 → 256 → 128 → 10",
        fmt(modelo_clf.count_params()),
        "Softmax (10 neurónios)",
        "sparse_categorical_crossentropy",
        f"{len(hist_clf['loss'])} ({melhor_epoca_clf})",
        fmt(len(hist_clf["loss"]) * math.ceil(len(X_treino_clf) / 128)),
        f"{tempo_clf:.0f} s ({tempo_clf / len(hist_clf['loss']):.1f} s por época)",
        f"acurácia = {res_clf['accuracy']:.2%}",
    ],
    "Regressão (California Housing)": [
        "valor contínuo (0.15 a 5.0)",
        f"{fmt(len(X_treino_reg))} / {fmt(len(X_val_reg))} / {fmt(len(X_teste_reg))}",
        "8 → 128 → 64 → 32 → 1",
        fmt(modelo_reg.count_params()),
        "Linear (1 neurónio)",
        "mse",
        f"{len(hist_reg['loss'])} ({melhor_epoca_reg})",
        fmt(len(hist_reg["loss"]) * math.ceil(len(X_treino_reg) / 64)),
        f"{tempo_reg:.0f} s ({tempo_reg / len(hist_reg['loss']):.2f} s por época)",
        f"MAE = {metricas_rede['MAE']:.3f} | RMSE = {metricas_rede['RMSE']:.3f} | R² =
{metricas_rede['R²']:.3f}",
    ],
}, index=["Alvo", "Treino / validação / teste", "Arquitetura", "Parâmetros treináveis",
          "Camada de saída", "Função de perda", "Épocas executadas (melhor)",
          "Atualizações dos pesos (passos)", "Tempo de treino", "Resultado no teste"])
resumo
```

[23]:

	Classificação (Fashion-MNIST)	Regressão (California Housing)
Alvo	10 classes (inteiros 0-9)	valor contínuo (0.15 a 5.0)
Treino / validação / teste	54 000 / 6 000 / 10 000	13 209 / 3 303 / 4 128
Arquitetura	784 → 256 → 128 → 10	8 → 128 → 64 → 32 → 1
Parâmetros treináveis	235 146	11 521
Camada de saída	Softmax (10 neurónios)	Linear (1 neurónio)
Função de perda	sparse_categorical_crossentropy	mse
Épocas executadas (melhor)	22 (16)	94 (74)
Atualizações dos pesos (passos)	9 284	19 458
Tempo de treino	63 s (2.9 s por época)	26 s (0.28 s por época)
Resultado no teste	acurácia = 89.16%	MAE = 0.340 | RMSE = 0.513 | R² = 0.800

Captura 8 *— tabela-resumo da comparação entre os dois modelos, gerada no notebook.*